

Overview of the Tool(s) Used

ChatGPT was the main tool used for generating code and helping with understanding of the assignment, occasionally googleAI was used for queries and copilot on visual code studio was used to help with commenting and other syntax fixes. I want to note that I have only pasted in the conversations with GPT which meaningfully improved my code or understanding. And all the conversations with ChatGPT was on the same thread, so assume that the GPT has context of the assignment, and my code at the each stage, just don't want this to be a million pages long.

How the Tool(s) Were Integrated in Your Workflow

I utilised ChatGPT to help generate code for repetitive things that are used for all assignments such as the code to read files and save input. Initially I needed something which could read the folder as an input, to save time for testing.

```
public static void main(String[] args) throws Exception {  
    if (args.length < 3) {  
        System.out.println("Usage: java A3 <frames> <quantum> <folder>");  
        return;  
    }  
  
    int totalFrames = Integer.parseInt(args[0]);  
    int quantum = Integer.parseInt(args[1]);  
    File folder = new File(args[2]);  
  
    File[] files = folder.listFiles((dir, name) -> name.toLowerCase().matches("process\\d+\\.txt"));  
    if (files == null || files.length == 0) {  
        System.out.println("No process files found in folder: " + folder);  
        return;  
    }  
    Arrays.sort(files, Comparator.comparing(File::getName));  
  
    List<Process> original = new ArrayList<>();  
    int pidCounter = 1;  
    for (File f : files) {  
        original.add(loadProcess(f, pidCounter++));  
    }  
}
```

```

        Simulator.simulateAndPrint(original, totalFrames, quantum, true); // Fixed-Local

        System.out.println("\n-----\n");

        Simulator.simulateAndPrint(original, totalFrames, quantum, false); // Variable-Global
    }

```

Since the requirements did ask for files to be inputted separately I did change this before submitting.

```

List<File> processFiles = new ArrayList<>();

for (int i = 2; i < args.length; i++) {
    File f = new File(args[i]);
    if (f.exists() && f.isFile()) {
        processFiles.add(f);
    } else {
        System.out.println("Warning: Skipping invalid file -> " + args[i]);
    }
}

// Sort by filename to ensure consistent process order (PID assignment)
processFiles.sort(Comparator.comparing(File::getName));

if (processFiles.isEmpty()) {
    System.out.println("No valid process files provided.");
    return;
}

// Load all process definitions
List<Process> original = new ArrayList<>();
int pidCounter = 1;
for (File f : processFiles) {
    original.add(loadProcess(f, pidCounter++));
}

// --- Run simulations ---

// Fixed-Local replacement policy
Simulator.simulateAndPrint(original, totalFrames, quantum, true);

```

```
System.out.println("\n-----\n");
```

```
// Variable-Global replacement policy
```

```
Simulator.simulateAndPrint(original, totalFrames, quantum, false);
```

```
}
```

GPT was also used to for errors and bugs which I had to fix throughout the program, such as my main issue where faults and turnaround were slightly off, using a prompt with the required results and mine I asked chat gpt to help fix it.

FIFO - Fixed-Local Replacement:

1 Process1 29 ...

2 Process2 33 ...

3 Process3 17 ...

4 Process4 32 ...

✗ *Your output*

Copy code

1 Process1 28 ...

2 Process2 30 ...

3 Process3 17 ...

4 Process4 34 ...

→ *meaning:*

Some page faults aren't delaying the total time as expected (e.g. Process1 ends too early).

Others (like Process4) end too late, which usually means they're not being rescheduled immediately after being unblocked.

⚙️ *Root cause*

Looking closely, the issue is this sequence inside your loop:

java

Copy code

```
if (!inMem) {
```

```
    cur.faultTimes.add(time);
```

```
    // ... handle replacement ...
```

```
    time += IO_DELAY;
```

```
    cur.unblockTime = time;
```

```

        blocked.add(cur);

        break; // move to next ready process
    }

```

and the unblocking logic:

java

Copy code

```

blocked.sort(Comparator.comparingInt(p -> p.unblockTime));
Iterator<Process> bit = blocked.iterator();
while (bit.hasNext()) {
    Process p = bit.next();
    if (p.unblockTime <= time) {
        ready.add(p);
        p.unblockTime = -1;
        bit.remove();
    } else break;
}

```

Here's what's off:

When a fault occurs, you add the IO delay ($time += IO_DELAY$), but you break immediately, so the process that just faulted never re-enters ready until the next top-of-loop iteration.

Meanwhile, the time skip logic for `ready.isEmpty()` only moves time to the minimum unblock time, not the exact unblocked moment if others are still ready.

The fix: we should

not advance time right away when faulting (just record $unblockTime = time + IO_DELAY$),

let the main loop's unblocking check handle the time progression cleanly.

That's what's causing the +/-1 turnaround difference between Fixed-Local and Variable-Global.

This fixed most of it apart from the sample 2 having the wrong results for variable global. In fixing this issue there was a lot of back and forth between ChatGPT. It offered a lot of solutions such as:

❑ **Root Cause**

In your simulation, you do this when a **page fault** occurs:

```
cur.unblockTime = time + IO_DELAY;
```

```
blocked.add(cur);
```

```
break; // move to next ready process
```

That's fine — but **you don't actually advance the global time** during that fault unless another process is ready.

Later, this piece tries to handle that:

```
if (ready.isEmpty()) {  
    if (!blocked.isEmpty()) {  
        int nextUnblock = blocked.stream()  
            .mapToInt(p -> p.unblockTime)  
            .min()  
            .orElse(time + 1);  
        time = nextUnblock;  
        continue;  
    } else break;  
}
```

So the global clock jumps to the next unblock time — skipping intermediate ticks.

That works for local replacement, where I/O events are nicely staggered between processes.

But in **global replacement**, processes fault more frequently and often in sync — so sometimes **the simulation jumps too far ahead**, making a process appear to finish earlier than it should (i.e. turnaround time shortens by a few ticks).

The Fix

Instead of **jumping** time forward, we should **increment it continuously** while processes are blocked — or at least advance it correctly when all are blocked.

You can fix this cleanly by replacing this block:

```
if (ready.isEmpty()) {  
    if (!blocked.isEmpty()) {  
        int nextUnblock = blocked.stream()  
            .mapToInt(p -> p.unblockTime)  
            .min()  
            .orElse(time + 1);  
        time = nextUnblock;  
        continue;  
    } else break;  
}
```

with this corrected version 

```

if (ready.isEmpty()) {
    if (!blocked.isEmpty()) {
        int nextUnblock = blocked.stream()
            .mapToInt(p -> p.unblockTime)
            .min()
            .orElse(time + 1);

        // Instead of jumping instantly, advance time properly
        time = Math.max(time + 1, nextUnblock);

        continue;
    } else break;
}

```

This ensures time **advances stepwise** when all processes are waiting — not an instantaneous skip.

Alternatively, for a slightly more realistic model, you can add this simpler and effective fix right after adding the process to blocked:

```

cur.unblockTime = time + IO_DELAY;
blocked.add(cur);

```

// In global mode, also advance time to reflect I/O initiation cost

```

if (!fixedLocal) time += 1;

```

```

break; // move to next ready process

```

This ensures that global I/O introduces at least 1 tick of real elapsed time (instead of jumping ahead later).

✓ After Fix — Expected Output for Sample2

After applying either fix above, your **FIFO – Variable Global Replacement** should match the target:

FIFO - Variable-Global Replacement:

PID	Process Name	Turnaround Time	# Faults	Fault Times
1	Process1	38	7	{0, 5, 10, 15, 20, 25, 31}
2	Process2	68	11	{0, 6, 11, 16, 21, 26, 37, 43, 49, 56, 62}
3	Process3	41	7	{0, 7, 12, 18, 23, 28, 36}
4	Process4	70	11	{0, 8, 13, 19, 24, 30, 37, 42, 48,

However for sample 2, fifo variable global replacement its supposed to be FIFO - Fixed-Local Replacement: PID Process Name Turnaround Time # Faults Fault Times 1 Process1 36 6 {0, 5, 10, 16, 22, 29} 2 Process2 70 11 {0, 6, 11, 17, 26, 37, 42, 47, 52, 59, 64} 3 Process3 28 3 {0, 7, 12} 4 Process4 76 12 {0, 8, 13, 21, 27, 35, 40, 45, 51, 60, 65, 71} -----

----- FIFO - Variable-Global Replacement: PID Process Name Turnaround Time # Faults Fault Times
1 Process1 30 5 {0, 5, 10, 16, 22} 2 Process2 60 8 {0, 6, 11, 17, 26, 36, 41, 46} 3 Process3 28 3 {0, 7, 12} 4 Process4 63 8 {0, 8, 13, 21, 27, 35, 40, 45} any ideas

ChatGPT said:

Ah! I see exactly what's going on — the difference between your **current simulation output** and the “expected” output is mostly in the **Variable-Global FIFO results**. Let's carefully analyze why.

1. Symptoms in your output

For Sample2, your current output shows:

FIFO - Variable-Global Replacement:

PID	Process Name	Turnaround Time	# Faults	Fault Times
1	Process1	38	7	{0, 5, 10, 15, 20, 25, 31}
2	Process2	68	11	{0, 6, 11, 16, 21, 26, 37, 43, 49, 56, 62}
3	Process3	41	7	{0, 7, 12, 18, 23, 28, 36}
4	Process4	70	11	{0, 8, 13, 19, 24, 30, 37, 42, 48, 57, 63}

Expected:

FIFO - Variable-Global Replacement:

PID	Process Name	Turnaround Time	# Faults	Fault Times
1	Process1	30	5	{0, 5, 10, 16, 22}
2	Process2	60	8	{0, 6, 11, 17, 26, 36, 41, 46}
3	Process3	28	3	{0, 7, 12}
4	Process4	63	8	{0, 8, 13, 21, 27, 35, 40, 45}

So the issues are:

1. **Turnaround times are too high** in your simulation.
2. **Page fault counts are higher** than expected.
3. **Process3's faults increased incorrectly.**

2. Likely causes

a) Global FIFO mismanagement

Your code currently does:

```
if (globalFIFO.size() >= totalFrames) {  
    int[] victim = globalFIFO.poll();  
    int vpid = victim[0], vpage = victim[1];  
    for (Process p : procs) if (p.pid == vpid) {  
        p.memory.remove(vpage);  
        Queue<Integer> newq = new LinkedList<>();
```

```

for (int v : p.fifoOrder) if (v != vpage) newq.add(v);

p.fifoOrder = newq;

break;
}
}

```

Problem:

- Every page fault triggers a **global check**, but you're **not correctly accounting for the IO delay**.
 - Your time variable is incremented **per page execution**, but when a page fault occurs and the process is blocked, **time isn't advanced correctly** in the global queue.
 - This can let multiple processes "pile up" on the same time slot, causing **extra page faults**.
-

b) Process unblocking logic

Currently, after adding a process to blocked:

```
cur.unblockTime = time + IO_DELAY;
```

```
blocked.add(cur);
```

```
break;
```

- You break out of the execution loop **without advancing time**.
 - In FIFO variable-global, the **oldest page may be replaced too early**, because multiple page faults can occur at the same time value.
-

c) Per-process executed count

- You increment executed **only if a page is in memory**, but when a page fault occurs, it **does not count toward the quantum**, which is correct.
 - But **blocked processes can get added back too soon** if your unblocking check happens **inside the inner loop**, leading to faster page access and extra faults.
-

3. How to fix

1. **Advance time properly after a page fault** in variable-global mode.
 - Right now, multiple processes may hit a page fault at the same time.
 - Add `time += 1` when a page fault is scheduled, so the next process sees the correct memory state.
2. **Update global FIFO correctly only after unblocking**
 - Only remove pages from memory when a page is actually replaced.
 - Ensure the global queue order is truly **FIFO across all processes**, not per process.
3. **Verify IO_DELAY handling**
 - Each page fault blocks a process for `IO_DELAY`.

- Make sure other processes that are ready **run in the meantime**, instead of letting blocked processes “stack up” and create extra page faults.

4. Quick pseudo-fix

Inside the main simulation loop:

```
if (!inMem) {  
    cur.faultTimes.add(time);  
  
    // Replace page if global memory full  
    if (!fixedLocal && globalFIFO.size() >= totalFrames) {  
        int[] victim = globalFIFO.poll();  
        // remove victim page from process memory  
        removeVictim(victim, procs);  
    }  
  
    cur.memory.add(pageNeeded);  
    cur.fifoOrder.add(pageNeeded);  
    if (!fixedLocal) globalFIFO.add(new int[] {cur.pid, pageNeeded});  
  
    // schedule IO delay  
    cur.unblockTime = time + IO_DELAY;  
    blocked.add(cur);  
  
    // Advance time to avoid multiple faults at same tick  
    time += 1;  
  
    break; // move to next ready process  
}
```

This ensures each page fault happens at a distinct time, and the global FIFO order is respected.

None of this worked however because the code was already perfect and I was stupid, and did not provide the machine with enough frames to actually calculate and achieve the best times. GPT also helped set up the output format.

Usefulness for Productivity and Time Saved

ChatGPT was super handy especially in the earlier stages by generating some base code which can take the files I was immediately able to get started on the actual problem and saved a lot of time with issues.

Weirdly though this time around it also wasted a lot of time, due to me relying on it to fix an issue which I thought was logic related but was human related. I would say that if I wasn't using AI I probably wouldn't have completely forgotten about inputs.

With the assignment, I would say that the AI had some trouble comprehending what needed doing, there was a lot of gaps I had to fill in for it. Such as each process starting at 0.

Ethical Considerations or Concerns in Using GenAI Tools

Out of the three assignments I have done for this course I would say this one probably showcases AI's biggest weakness, which is its understanding when the problem is large and complex, (split in multiple files, lots of requirements for the results). Even with this it was extremely useful and helped extensively with the completion and quality of the assignment. Concerns wise I think it's important to not be stuck into a loop of sending GPT your output and asking why it's wrong and waiting for new code. I know I was for a good bit, if I had just taken second to look at the input command, I was typing it would've saved me hours.